Quarto Project

Joshua Tanyi

2026-04-13

Section 1: Busiest Airports Analysis

1.1: Introduction

In this section, my main goal analyze passenger traffic data for six of the busiest airports. Within this analysis a main subgoal is to compare trends over time and identify any patterns across airports.

1.2: Table

As shown in Table 1 passenger traffic varies across six of the busiest airports in the world from the year 2020 to the year 2025.

Table 1: Passenger traffic (in millions) for the six busiest airports from 2020 to 2025

Airport	2020	2021	2022	2023	2024	2025
Beijing Daxing International Airport	NA	25.0510	NA	NA	49.4410	53.6189
Dallas Fort Worth International Airport	39.3650	62.4658	73.3629	81.7555	87.8179	85.6601
Dubai International Airport	25.8368	29.1106	66.0700	86.9944	92.3315	95.2000
Frankfurt Airport	18.7686	24.8128	48.9185	59.3554	61.5650	63.1897
Tokyo Haneda Airport	31.0552	25.8764	50.2907	78.7193	85.9002	91.4300

1.2.1: Analysis of the table

As shown in Table 1, passenger traffic varies across both airports and years. Some airports, such as Dallas Fort Worth and Dubai International, show steady increases in passenger numbers over time, suggesting consistent growth in travel demand. Other airports exhibit more

fluctuation, which may reflect changes in global travel patterns or external factors affecting certain regions.

Additionally, while all six airports are among the busiest in the world, there are noticeable differences in scale, with some airports consistently handling higher passenger volumes than others. Overall, the table highlights both long-term growth trends and variation in passenger traffic across major international airports.

1.3: Visualizaltion

Figure 1 shows the varying numbers in passenger traffic across 6 of the busiest airports from 2020 - 2025

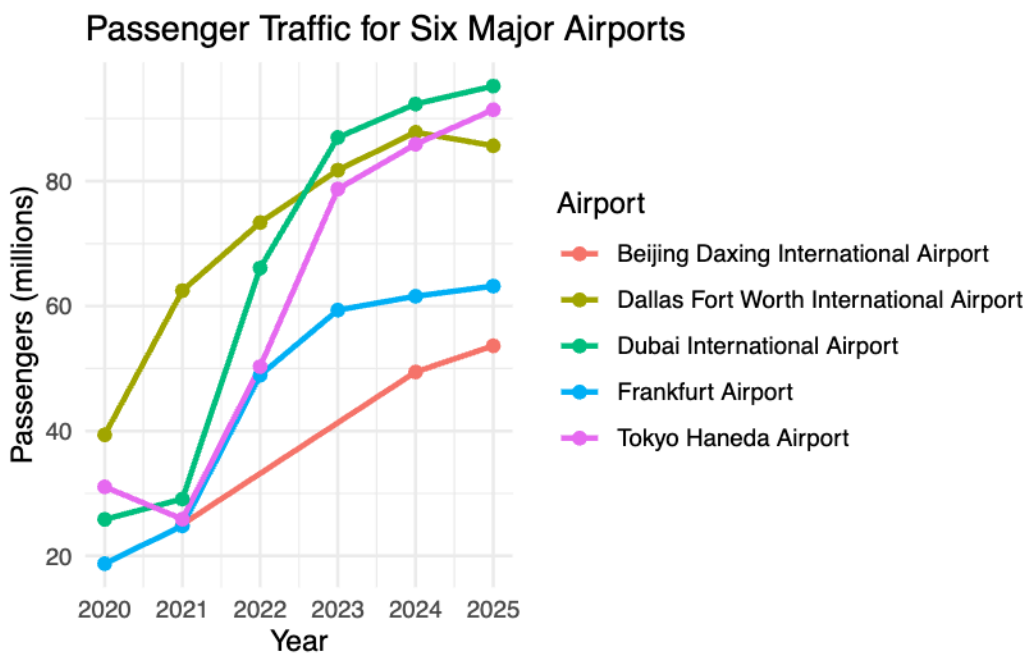

Figure 1: Passenger traffic trends (in millions) for the six busiest airports from 2020 to 2025

1.3.1: Analysis of Visualization

As shown in Figure 1, passenger traffic trends differ across the six airports over time. Several airports, such as Dallas Fort Worth and Dubai International, show consistent growth from 2020 to 2025, indicating increasing travel demand. In contrast, other airports display more fluctuation, which reflects changes in global travel patterns during certain years. The visualization makes these differences easier to observe compared to the table, as the overall direction

and steepness of each line clearly show how passenger traffic changes over time. This highlights both steady growth for some airports and variability for others.

Section 2: Monte Carlo Numerical Integration

2.1 Introduction

In this section, I use the Monte Carlo method to estimate the value of an integral. This method uses random sampling to approximate the area under a curve. By comparing points that fall below the function to the total number of sampled points, I can estimate the proportion of the rectangle that lies under the curve.

2.2 Visualization

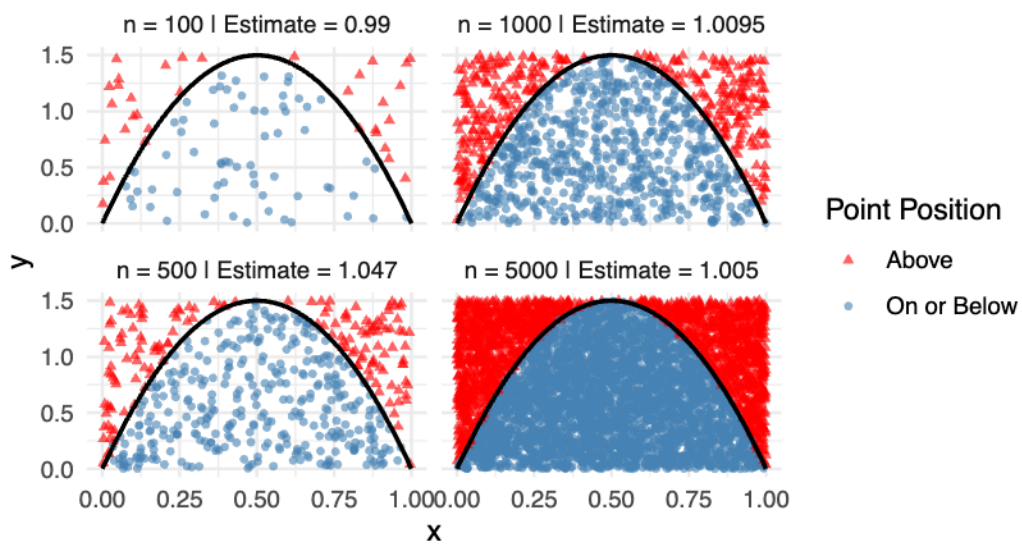

Figure 2: Monte Carlo approximation of the integral of a density using different sample sizes

> **Long Description**
>
> Figure 2 is a four-panel Monte Carlo visualization for estimating the integral of a Beta(2, 2) density on the interval from 0 to 1. Each panel shows a different sample size: 100, 500, 1000, and 5000. In every panel, random points are generated inside a rectangle with x-values from 0 to 1 and y-values from 0 to 1.5. A smooth black curve represents the Beta(2, 2) density.

Points are classified according to whether they fall above the curve or on or below it. These groups are distinguished using both color and shape. Each panel also displays a Monte Carlo estimate of the integral. As the sample size increases, the points fill the rectangle more evenly and the estimates become more stable. The figure suggests that the estimates are approaching 1 as the number of simulated points increases.

Analysis

As shown in Figure 2, the Monte Carlo method estimates the integral by comparing the number of random points that fall on or below the curve to the total number of points generated in the rectangle. The use of both color and shape makes it easier to distinguish the two groups of points and understand how the estimate is being calculated. The utilization of different panels show that sample size has an important effect on the approximation. When the sample size is smaller, the points are more sparse and the estimate varies more compared to when the sample size is large. A larger sample sizes makesthe points cover the region more evenly, making the estimate more stable and closer to the true value of the integral. Since the function is a probability density function on the interval [0,1], the exact value of the integral is 1. The estimates shown in the larger sample size panels appear to approach this value, which supports the conclusion that the true answer is 1.

Section 3: Comparing AI generated prompt with a plan vs generic

3.1 AI GENERATED

3.1.1 Goal

The goal of this analysis is to transform the calcium dataset into a tidy structure and develop a visualization that clearly shows and compares trends in calcium levels over time between the null and diet/exercise treatment groups.

3.1.2 Needs

The dataset calcium.csv A data analysis environment such as R Libraries Data wrangling tools Visualization tools Understanding of tidy data principles: Awareness of the dataset structure: A plan to check and verify the data after transformation.

3.1.3 Steps

Steps:

1. Load and inspect the data

- Import calcium.csv into the environment
- View the dataset structure
- Confirm that the data is in wide format with separate columns for each group and time point
- Ensure there are 8 measurement columns

2. Reshape the data from wide to long format

- Use pivot_longer() to combine all measurement columns into two columns:
- one for calcium values
- one for combined labels (group + time)

3. Extract and create meaningful variables

- Create a Group variable (Null vs Treatment)
- Create a Year variable (Initial, Year 1, Year 2, Year 3)
- Convert initial to zero
- Convert year values into numeric form
- Ensure only two groups exist and Year values are 0–3

4. Clean and verify the data

- Check for missing or incorrect values
- Ensure each row represents one observation (one subject at one time point)
- Confirm expected structure: Columns: Group, Year, Calcium

5. Create the visualization

- Use ggplot() to build a line plot: x-axis = Year y-axis = Calcium color = Group
- Add: geom_line() to show trends geom_point() to show actual values Optionally compute and plot group averages for clearer comparison Ensure both groups are clearly visible and distinguishable

6. Interpret the results

- Compare how calcium levels change over time in each group
- Identify any differences in trends between the null and treatment groups
- Note whether the treatment group shows improvement or slower decline
- Verify interpretation matches what is shown in the graph

7. Refine and improve the output

- Add clear titles and axis labels
- Make sure the visualization is easy to read and understand
- Make adjustments if anything is unclear or misleading

4.1.4 AI Generated output(my plan)

```
#AI GENERATED CODE
# Load libraries
library(tidyverse)

# 1. Load and inspect the data
calcium <- read.csv("calcium.csv")

str(calcium)
colnames(calcium)
head(calcium)

# 2. Reshape from wide to long format
calcium_long <- calcium %>%
  pivot_longer(
    cols = -ID,
    names_to = "Group_Time",
    values_to = "Calcium"
  )

# 3. Extract Group and Year variables
calcium_tidy <- calcium_long %>%
  separate(Group_Time, into = c("Group", "Year"), sep = "_") %>%
  mutate(
    Group = case_when(
      Group == "null" ~ "Null",
      Group == "trt" ~ "Treatment"
    ),
    Year = as.numeric(Year)
  )

# 4. Clean and verify the data
sum(is.na(calcium_tidy))        # Check missing values
glimpse(calcium_tidy)           # Inspect structure
```

```r
calcium_tidy %>%
  count(Group, Year)            # Verify distribution

# 5. Create summary for visualization
calcium_summary <- calcium_tidy %>%
  group_by(Group, Year) %>%
  summarise(
    mean_calcium = mean(Calcium, na.rm = TRUE),
    .groups = "drop"
  )

# 6. Visualization (final plot)
ggplot(calcium_summary, aes(x = Year, y = mean_calcium, color = Group)) +
  geom_line(size = 1.3) +
  geom_point(size = 3) +
  scale_x_continuous(breaks = 0:3) +
  labs(
    title = "Comparison of Calcium Trends Over Time",
    subtitle = "Null vs Treatment Groups",
    x = "Year (0 = Initial)",
    y = "Average Calcium Level",
    color = "Group"
  ) +
  theme_minimal(base_size = 14)
```

4.1.5: AI Generated output(generic)

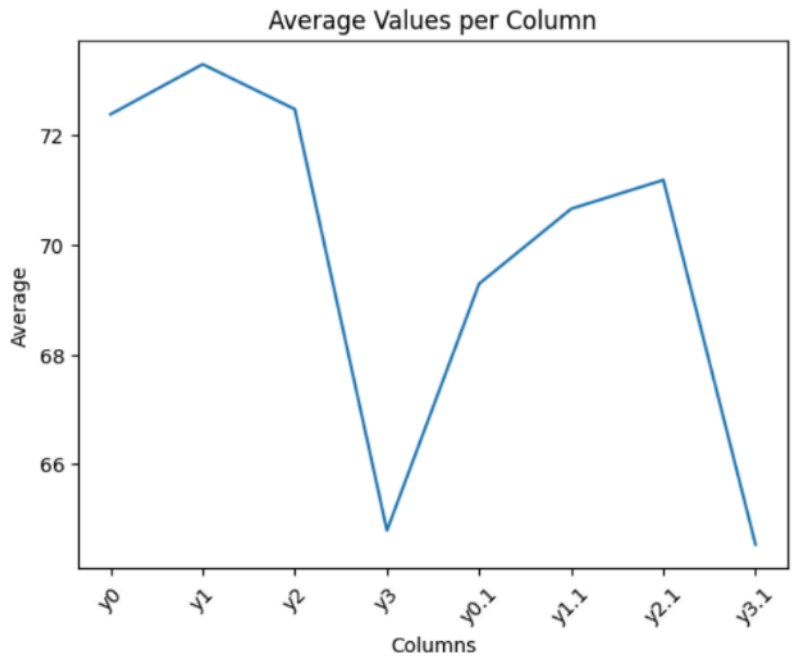

Figure 3: Figure 3: ChatGPT-generated visualization of average values per column

Figure 3 shows the visualization produced by ChatGPT when given a generic prompt

Comparison.

The prompt with the plan included goes into much detail providing code which will wrangle and properly tidy the data and produce a visual which clearly shows variation with properly labelled captions and labels whereas the generic just shows us a vague plot which is pretty difficult to understand and no clear proper information presented

Section 4: Reflection

Over the course of this course, I have learned many things about programming in the language of r. I learned data wrangling, how to use the PCIP style to create code in R. I also learned how to visualize data using ggplot. I learned how to use libraries, what they were, packages and how all that ties into making data more readable and accessible for others to use.I also developed a much stronger understanding of how to work with data in a structured and meaningful way. Before, I would look at datasets without really thinking about their structure, but now

I understand that each row should represent a single observation and each column should represent a variable.

For example, in this assignment specifically section 3 regarding the calcium csv, I had to transform the data from a wide format into a long format. At first, this was confusing because I did not fully understand how the column names contained multiple pieces of information. However, after working through it, I was able to separate the group (null vs treatment) and time (Initial, Year 1, Year 2, Year 3) into their own variables. This helped me clearly organize the data into Group, Year, and Calcium columns.

Another great skill I developed was creating visualizations which had meaning. Instead of just plotting raw data, I learned how to summarize the data and create a line plot that shows variation and trends over time.

I also learned the importance of verifying my work. There were times when my code did not produce useful results, which made me to go back and check the data structure, and ensure everything was correct

Overall, I feel that I have made solid improvements towards not only in writing code, but also in thinking more critically about data, structure, and interpretation. I now understand most ways how each step in the analysis process connects, from cleaning the data to creating a final visualization that communicates a clear message.

Code Appendix

```r
#WRANGLING AIRPORT DATA
# load packages
library(rvest)
library(tidyverse)
library(kableExtra)
library(scales)

url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

airports_keep <- c(
  "Hartsfield-Jackson Atlanta International Airport",
  "Frankfurt Airport",
  "Beijing Daxing International Airport",
  "Dubai International Airport",
  "Tokyo Haneda Airport",
  "Dallas Fort Worth International Airport"
)

tables <- read_html(url) |>
  html_elements("table.wikitable") |>
  html_table()

table_2025 <- tables[[1]]
table_2024 <- tables[[2]]
table_2023 <- tables[[3]]
table_2022 <- tables[[4]]
table_2021 <- tables[[5]]
table_2020 <- tables[[6]]

data_2025 <- table_2025 |> select(Airport, Totalpassengers) |> mutate(year = 2025)
data_2024 <- table_2024 |> select(Airport, Totalpassengers) |> mutate(year = 2024)
data_2023 <- table_2023 |> select(Airport, Totalpassengers) |> mutate(year = 2023)
data_2022 <- table_2022 |> select(Airport, Totalpassengers) |> mutate(year = 2022)
data_2021 <- table_2021 |> select(Airport, Totalpassengers) |> mutate(year = 2021)
data_2020 <- table_2020 |> select(Airport, Totalpassengers) |> mutate(year = 2020)

airport_data <- bind_rows(
  data_2025, data_2024, data_2023,
  data_2022, data_2021, data_2020
```

```r
)

airport_data <- airport_data |>
  rename(airport = Airport, passengers = Totalpassengers) |>
  mutate(
    airport = str_remove(airport, "^\\d+\\.\\s*"),
    airport = str_remove_all(airport, "\\[.*?\\]"),
    airport = str_squish(airport),
    passengers = str_remove_all(passengers, "\\[.*?\\]"),
    passengers = str_remove_all(passengers, ","),
    passengers = as.numeric(passengers)
  ) |>
  filter(airport %in% airports_keep) |>
  select(year, airport, passengers) |>
  arrange(airport, year)

options(scipen = 999)
#CREATING TABLE FOR AIRPORT DATA
airport_table <- airport_data |>
  mutate(passengers = passengers / 1e6) |>   # convert to millions
  pivot_wider(
    names_from = year,
    values_from = passengers
  ) |>
  select(airport, `2020`, `2021`, `2022`, `2023`, `2024`, `2025`) |>
  arrange(airport) |>
  rename(Airport = airport) |>
  mutate(across(-Airport, ~round(., 4)))     # limit to 4 decimal places

knitr::kable(airport_table)
#PLOTTING AIRPORT DATA
ggplot(airport_data, aes(x = year, y = passengers / 1e6, color = airport)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    title = "Passenger Traffic for Six Major Airports",
    x = "Year",
    y = "Passengers (millions)",
    color = "Airport"
  ) +
  theme_minimal()
#WRANGLING MONTE CARLO DATA AND PLOTTING
```

```r
library(tidyverse)

# function to simulate random points in the rectangle
monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {
  data.frame(
    x = runif(n, min = x_min, max = x_max),
    y = runif(n, min = y_min, max = y_max)
  )
}

# rectangle area
rect_area <- (1 - 0) * (1.5 - 0)

# sample sizes for small multiples
sample_sizes <- c(100, 500, 1000, 5000)

# generate simulation data for each sample size
mc_data <- purrr::map_dfr(sample_sizes, function(n) {

  points_ou <- monte_carlo_points(
    n = n,
    x_min = 0,
    x_max = 1,
    y_min = 0,
    y_max = 1.5
  )

  points_ou <- points_ou |>
    mutate(
      position = if_else(
        y <= dbeta(x, shape1 = 2, shape2 = 2),
        "On or Below",
        "Above"
      )
    )

  pp_under <- mean(points_ou$position == "On or Below")
  estimate <- pp_under * rect_area

  points_ou |>
    mutate(
      sample_size = paste("n =", n),
```

```r
      estimate = round(estimate, 4)
    )
})

ggplot(mc_data, aes(x = x, y = y)) +
  geom_point(
    aes(color = position, shape = position), # distinguish points above the
    alpha = 0.6,                             #curve from those on or below it
    size = 1.2
  ) +
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2), # overlay the curve
    xlim = c(0, 1),
    linewidth = 0.8
  ) +
  coord_cartesian(
    xlim = c(0, 1),
    ylim = c(0, 1.5)     # restrict the plot to the simulation rectangle
  ) +
  facet_wrap(
    ~ paste(sample_size, "| Estimate =", estimate)  # create small multiples
  ) +
  labs(
    x = "x",
    y = "y",
    color = "Point Position",
    shape = "Point Position"
  ) +
  scale_color_manual(
    values = c(
      "On or Below" = "steelblue",
      "Above" = "red"
    )                                        # assign colors
  ) +
  scale_shape_manual(
    values = c(
      "On or Below" = 16,
      "Above" = 17
    )                             # assign different shapes
  ) +
  theme_minimal()
```

```r
#AI GENERATED CODE
# Load libraries
library(tidyverse)

# 1. Load and inspect the data
calcium <- read.csv("calcium.csv")

str(calcium)
colnames(calcium)
head(calcium)

# 2. Reshape from wide to long format
calcium_long <- calcium %>%
  pivot_longer(
    cols = -ID,
    names_to = "Group_Time",
    values_to = "Calcium"
  )

# 3. Extract Group and Year variables
calcium_tidy <- calcium_long %>%
  separate(Group_Time, into = c("Group", "Year"), sep = "_") %>%
  mutate(
    Group = case_when(
      Group == "null" ~ "Null",
      Group == "trt" ~ "Treatment"
    ),
    Year = as.numeric(Year)
  )

# 4. Clean and verify the data
sum(is.na(calcium_tidy))        # Check missing values
glimpse(calcium_tidy)           # Inspect structure

calcium_tidy %>%
  count(Group, Year)            # Verify distribution

# 5. Create summary for visualization
calcium_summary <- calcium_tidy %>%
  group_by(Group, Year) %>%
  summarise(
    mean_calcium = mean(Calcium, na.rm = TRUE),
```

```r
    .groups = "drop"
  )

# 6. Visualization (final plot)
ggplot(calcium_summary, aes(x = Year, y = mean_calcium, color = Group)) +
  geom_line(size = 1.3) +
  geom_point(size = 3) +
  scale_x_continuous(breaks = 0:3) +
  labs(
    title = "Comparison of Calcium Trends Over Time",
    subtitle = "Null vs Treatment Groups",
    x = "Year (0 = Initial)",
    y = "Average Calcium Level",
    color = "Group"
  ) +
  theme_minimal(base_size = 14)
```

GenAI Usage Appendix

Use 1

Tool: ChatGPT (GPT-5.3)

Date Used: April 14, 2026

Prompt: ### 3.1.1 Goal

The goal of this analysis is to transform the calcium dataset into a tidy structure and develop a visualization that clearly shows and compares trends in calcium levels over time between the null and diet/exercise treatment groups.

3.1.2 Needs

The dataset calcium.csv A data analysis environment such as R Libraries Data wrangling tools Visualization tools Understanding of tidy data principles: Awareness of the dataset structure: A plan to check and verify the data after transformation.

3.1.3 Steps

Steps:

1. Load and inspect the data

- Import calcium.csv into the environment
- View the dataset structure
- Confirm that the data is in wide format with separate columns for each group and time point
- Ensure there are 8 measurement columns

2. Reshape the data from wide to long format

- Use pivot_longer() to combine all measurement columns into two columns:
- one for calcium values
- one for combined labels (group + time)

3. Extract and create meaningful variables

- Create a Group variable (Null vs Treatment)

- Create a Year variable (Initial, Year 1, Year 2, Year 3)
- Convert initial to zero
- Convert year values into numeric form
- Ensure only two groups exist and Year values are 0–3

4. Clean and verify the data

- Check for missing or incorrect values
- Ensure each row represents one observation (one subject at one time point)
- Confirm expected structure: Columns: Group, Year, Calcium

5. Create the visualization

- Use ggplot() to build a line plot: x-axis = Year y-axis = Calcium color = Group
- Add: geom_line() to show trends geom_point() to show actual values Optionally compute and plot group averages for clearer comparison Ensure both groups are clearly visible and distinguishable

6. Interpret the results

- Compare how calcium levels change over time in each group
- Identify any differences in trends between the null and treatment groups
- Note whether the treatment group shows improvement or slower decline
- Verify interpretation matches what is shown in the graph

7. Refine and improve the output

- Add clear titles and axis labels
- Make sure the visualization is easy to read and understand
- Make adjustments if anything is unclear or misleading. Carry out this plan give me the code in one chunk

Response:

Load libraries library(tidyverse)

- 1. Load and inspect the data calcium <- read.csv("calcium.csv")

str(calcium) colnames(calcium) head(calcium)

- 2. Reshape from wide to long format calcium_long <- calcium %>% pivot_longer(cols = -ID, names_to = "Group_Time", values_to = "Calcium")
- 3. Extract Group and Year variables calcium_tidy <- calcium_long %>% separate(Group_Time, into = c("Group", "Year"), sep = "_") %>% mutate(Group = case_when(Group =="null" ~ "Null", Group == "trt" ~ "Treatment"), Year = as.numeric(Year))

- 4. Clean and verify the data sum(is.na(calcium_tidy)) # Check missing values glimpse(calcium_tidy) # Inspect structure

calcium_tidy %>% count(Group, Year) # Verify distribution

- 5. Create summary for visualization calcium_summary <- calcium_tidy %>% group_by(Group, Year) %>% summarise(mean_calcium = mean(Calcium, na.rm = TRUE), .groups = "drop")
- 6. Visualization (final plot) ggplot(calcium_summary, aes(x = Year, y = mean_calcium, color = Group)) + geom_line(size = 1.3) + geom_point(size = 3) + scale_x_continuous(breaks = 0:3) + labs(title = "Comparison of Calcium Trends Over Time", subtitle = "Null vs Treatment Groups", x = "Year (0 = Initial)", y = "Average Calcium Level", color = "Group") + theme_minimal(base_size = 14)

Use 2 (provided)

Prompt: Create a data visualization for these data. Calcium.csv

I created a visualization showing the average value of each column in your dataset. What this shows: • Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). • You can quickly compare which variables tend to have higher or lower values overall. • For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create: • Boxplots (to show spread and outliers) • Time series plots (if rows represent time/order) • Correlation heatmaps • Side-by-side comparisons (e.g., y0 vs y0.1) Just tell me Prompt: What is your version information? You're chatting with ChatGPT based on the GPT-5.3 architecture. Key details: • Model: GPT-5.3 • Knowledge cutoff: August 2025 • Current date (system): March 31, 2026 If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know (This chat took place on Tuesday, March 31st, 2026.)

Use 3: Long Description Assistance

- Tool: ChatGPT (GPT-5.3)
- Date: April 2026

Prompt: "Write a long description for a Monte Carlo simulation plot showing random points under a Beta(2,2) curve." ***Response:*** Figure 2 is a four-panel Monte Carlo visualization for estimating the integral of a Beta(2, 2) density on the interval from 0 to 1. Each panel shows a different sample size: 100, 500, 1000, and 5000. In every panel, random points are generated inside a rectangle with x-values from 0 to 1 and y-values from 0 to 1.5. A smooth black curve represents the Beta(2, 2) density.

Points are classified according to whether they fall above the curve or on or below it. These groups are distinguished using both color and shape. Each panel also displays a Monte Carlo estimate of the integral. As the sample size increases, the points fill the rectangle more evenly and the estimates become more stable. The figure suggests that the estimates are approaching 1 as the number of simulated points increases.